SIMATS ENGINEERING

CO4 ASSESSMENT TOOL 01

COURSE NAME: Ethical Hacking **COURSE CODE: ITA1416**

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.

QUESTIONS: 01 **Total Marks:100**

Annexure A

SECURITY TESTING SIMULATION TASK

Code of Conduct:

I **G.RAJU (192411065)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

G.RAJU

Signature of the student:

S.No.	Assessment Criterion	Excellent	Good	Satisfactory	Needs Improvement	Marks
1	Tool Selection and Configuration(20)	Selects appropriate open-source tools, configures them correctly, and clearly explains the purpose of each tool.	Selects suitable tools and configures them with minor errors or limited explanation.	Uses basic tools with partial configuration and limited understanding.	Selects inappropriate tools or fails to configure them correctly.	
2	Testing Procedure and Execution(20)	Follows a systematic, authorized, safe, and well-documented testing procedure within the approved scope.	Performs the major testing steps correctly with minor omissions.	Completes only basic testing steps with limited documentation.	Testing procedure is incomplete, unsafe, unauthorized, or poorly executed.	
3	Identification and Validation of Vulnerabilities (20)	Accurately identifies vulnerabilities, manually validates findings, and distinguishes confirmed issues from false positives.	Identifies and validates most important findings with minor gaps.	Identifies basic vulnerabilities but provides limited validation.	Findings are inaccurate, unsupported, or based only on automated tool output.	
4	Risk Analysis and Remediation(20)	Clearly explains the cause, likelihood, impact, severity, and practical remediation for every confirmed vulnerability.	Provides suitable risk analysis and recommendations with minor omissions.	Provides general risk descriptions and basic remediation measures.	Risk ratings are unsupported or recommendations are impractical or unrelated.	
5	Evidence, Report Quality, and Ethical Compliance(20)	Provides clear commands, screenshots, outputs, tables, references, and a professional report while maintaining authorization, confidentiality, and ethical conduct.	Provides sufficient evidence and a well-organized report with minor presentation issues.	Provides limited evidence or an adequately organized report.	Evidence is missing, the report is unclear, or ethical and legal requirements are ignored.	
					Total	

Question 1: Web Server Reconnaissance and Vulnerability Analysis

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for a security assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and web-server vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directories and files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendations.

AIM

To perform reconnaissance and vulnerability assessment of an authorized vulnerable web server using Google, WHOIS, Ping, Tracert, IPConfig, Netstat, Nmap, WhatWeb, Curl, Nikto, and Gobuster/Dirb, and to identify security weaknesses and information disclosure.

TOOLS USED

- Google
- WHOIS
- Ping
- Tracert
- IPConfig
- Netstat
- Nmap
- WhatWeb
- Curl
- Wget
- Nikto
- Gobuster / Dirb

PROCEDURE

1. Gather publicly available information about the authorized target using Google and WHOIS.
2. Perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat.
3. Use Nmap to identify open ports, services, and service versions.
4. Use WhatWeb and HTTP-header analysis to identify web technologies and information disclosure.
5. Perform authorized web-server scanning and directory enumeration using Nikto and Gobuster/Dirb, analyze the findings, and recommend suitable security controls.

COMMANDS WHOIS

whois <authorized-domain>

Ping ping <target-ip>

Tracert / Traceroute

Windows:

tracert <target>

Kali Linux:

traceroute <target>

IPConfig

Windows:

ipconfig

Kali Linux:

ip addr

Netstat

netstat -ano

Nmap

nmap -sC -sV <target-ip>

WhatWeb

whatweb http://<target-ip>

Curl – HTTP Headers

curl -I http://<target-ip>

Wget

wget --server-response --spider http://<target-ip> **Nikto**

nikto -h http://<target-ip> **Gobuster** gobuster dir -u

http://<target-ip> -w <authorized-wordlist> **Dirb** dirb

http://<target-ip>

OBSERVATION

The reconnaissance phase provides information about the target's network location, DNS information, open ports, web technologies, server headers, and publicly accessible resources.

Record the actual findings:

Category	**Observation**
Domain information	<observation>
IP address	<IP>
Open ports	<ports>
Services	<services>
Web technologies	<technologies>
HTTP headers	<headers>
Web directories	<directories>
Potential vulnerabilities	<findings>

FINDINGS AND SEVERITY

Finding	**Severity**	**Impact**
Unnecessary open service	Medium	Increases attack surface
Server/version disclosure	Low	Helps technology fingerprinting
Exposed backup/configuration file	High	May disclose sensitive information
Unnecessary administrative page	High	May increase unauthorizedaccess risk
Outdated web component	High	May contain known security weaknesses

RECOMMENDATIONS

1. Disable unnecessary services and ports.
2. Remove backup, temporary, and configuration files from public directories.
3. Restrict administrative interfaces.
4. Update the web server and application components.
5. Minimize unnecessary server information in HTTP headers.
6. Configure appropriate security headers.
7. Disable directory listing where unnecessary.
8. Apply firewall and access-control rules.
9. Regularly perform vulnerability assessments.

RESULT

Successfully performed reconnaissance and vulnerability assessment of the authorized web server. Network exposure, web technologies, HTTP information disclosure, and potentially vulnerable resources were identified and appropriate hardening measures were recommended.

Question 2: Web Application Vulnerability Assessment

A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

AIM

To assess an authorized vulnerable web application using Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools and identify common web-application security weaknesses.

TOOLS USED

- Burp Suite
- OWASP ZAP
- Nikto
- Nmap
- Curl
- Browser Developer Tools

PROCEDURE

1. Identify the application's technologies, pages, parameters, cookies, and sessions.
2. Capture normal HTTP requests using Burp Suite or OWASP ZAP.
3. Analyze requests and responses to understand the application's attack surface.
4. Perform automated security assessment using authorized scanning tools and safely validate selected findings.
5. Document confirmed findings, severity, impact, and remediation recommendations.

COMMANDS Nmap

nmap -sC -sV <target-ip>

Curl

curl -I http://<target-ip>

Nikto nikto -h

http://<target-ip>

TOOL CONFIGURATION

Burp Suite:

- Start Burp Suite.
- Configure the browser proxy to use the Burp proxy.
- Browse the authorized application.
- Capture and inspect HTTP requests and responses.
- Review parameters, cookies, headers, and session information.

OWASP ZAP:

- Start ZAP.

- Configure the authorized application as the target.
- Use passive scanning during normal browsing.
- Run the authorized automated assessment.
- Review alerts and manually validate selected findings safely.

OBSERVATION

The assessment should examine:

- Input fields
- URL parameters
- Cookies
- Session identifiers
- Authentication mechanisms
- Authorization controls
- HTTP methods
- Security headers
- Error messages
- Directory and resource exposure

VULNERABILITY ANALYSIS

Vulnerability	Severity	Potential Impact
SQL Injection	High/Critical	Unauthorized database access or data manipulation
XSS	Medium/High	Malicious script execution in a user's browser
Weak authentication	High	Unauthorized account access
Session weakness	High	Possible account/session compromise
Directory traversal	High	Unauthorized file access
Security misconfiguration	Medium	Increased attack surface or information disclosure

RECOMMENDATIONS

1. Use parameterized queries and prepared statements.
2. Validate and sanitize user input.
3. Implement secure authentication.
4. Use secure session management.
5. Apply server-side authorization checks.

6. Configure appropriate security headers.
7. Disable unnecessary services and features.
8. Avoid detailed error messages in production.
9. Keep application dependencies updated.
10. Perform regular security testing.

RESULT

The authorized web application was assessed using reconnaissance and security-testing tools. Potential vulnerabilities were documented according to their severity and impact, and secure coding and configuration recommendations were provided.

Question 3: Security Header and HTTPS Configuration Assessment

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and X-ContentType-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificaterelated issues. Analyze the security purpose and potential impact of each misconfiguration, classify the findings according to their severity, and recommend appropriate HTTPS, TLS, certificate, and security-header hardening measures. Submit the commands used, screenshots, observations, identified risks, severity classification, and remediation recommendations.

AIM

To assess the HTTPS/TLS configuration and HTTP security headers of an authorized web application using Curl and Nmap SSL scripts.

PROCEDURE

1. Open Kali Linux Terminal.
2. Identify the authorized application's HTTP and HTTPS services.
3. Use Curl to inspect HTTP response headers.
4. Use Nmap SSL scripts to analyze the TLS configuration.
5. Check redirects, security headers, protocols, cipher configuration, and certificate information.

COMMANDS

HTTP Headers

curl -I http://<target>

HTTPS Headers

curl -I https://<target> **Follow**

HTTP Redirects

curl -I -L http://<target>

Nmap SSL Enumeration

nmap -p 443 --script ssl-enum-ciphers,ssl-cert <target-ip>

HEADERS TO CHECK

Header	Security Purpose
Content-Security-Policy	Helps reduce XSS and content-injection risks
Strict-Transport-Security	Enforces HTTPS connections
X-Frame-Options	Helps prevent clickjacking
X-Content-Type-Options	Helps prevent MIME-type sniffing

OBSERVATION

Record the actual headers returned by the application:

Security Header	Present?	Configuration	Severity
CSP	Yes/No	<value>	<severity>
HSTS	Yes/No	<value>	<severity>
X-Frame-Options	Yes/No	<value>	<severity>
X-Content-Type-Options	Yes/No	<value>	<severity>

TLS ANALYSIS

The Nmap SSL scan should be examined for:

- Supported TLS versions
- Weak protocols
- Cipher suites

- Certificate validity
- Certificate expiration
- Certificate hostname
- Certificate chain

RISK CLASSIFICATION

Finding	Severity	Impact
Missing CSP	Medium	Increased client-side injection risk
Missing HSTS	Medium	Reduced HTTPS enforcement
Missing X-Frame-Options	Low/Medium	Increased clickjacking risk
Missing X-Content-TypeOptions	Low	Increased MIME-sniffing risk
Weak TLS protocol	High	Reduced transport security
Weak cipher suite	High/Medium	Potentially weaker encrypted communication
Invalid/expired certificate	High	Trust and authentication problems

RECOMMENDATIONS

1. Enable HTTPS for sensitive communication.
2. Configure HSTS appropriately.
3. Implement a suitable Content-Security-Policy.
4. Configure X-Frame-Options or an appropriate CSP frame-ancestors policy.
5. Enable X-Content-Type-Options.
6. Disable obsolete TLS protocols.
7. Use strong TLS cipher suites.
8. Maintain valid certificates.
9. Redirect HTTP traffic to HTTPS where appropriate.
10. Regularly review TLS configuration.

RESULT

Successfully assessed the authorized application's HTTP security headers and HTTPS/TLS configuration and identified configuration weaknesses requiring remediation.

Question 4: Broken Access-Control Assessment

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures, classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity, impact, and remediation measures.

AIM

To assess access-control mechanisms in an authorized student portal using OWASP ZAP and determine whether users can access resources outside their permitted roles.

PROCEDURE

1. Start the authorized student portal in the isolated laboratory environment.
2. Log in using the test student account provided by the faculty.
3. Capture normal requests using OWASP ZAP.
4. Identify test object identifiers and administrator-only resources provided for the exercise.
5. Compare authorized and unauthorized responses and document only confirmed accesscontrol weaknesses.

TOOL USED OWASP

ZAP

TESTING PROCESS

1. Start ZAP and configure the browser proxy.
2. Log in using the authorized student account.
3. Browse student records and capture the requests.
4. Identify the object/reference identifiers supplied for the lab.

5. Compare responses for authorized and unauthorized test cases.
6. Check whether the server performs authorization checks rather than relying only on clientside restrictions.

OBSERVATION

Test Case	Expected Result	Actual Result	Status
Student accesses own record	Access allowed	<result>	<status>
Student accesses another test record	Access denied	<result>	<status>
Student accesses administrator resource	Access denied	<result>	<status>
Administrator accesses authorized resource	Access allowed	<result>	<status>

AUTHENTICATION VS AUTHORIZATION

Authentication determines who the user is.

Authorization determines what the authenticated user is allowed to access.

For example, successfully logging in as a student does not mean that the student should be able to access administrator-only resources.

RISK CLASSIFICATION

Finding	Severity	Impact
Access to another user's record	High	Unauthorized disclosure of personal data
Student access to admin resource	Critical/High	Privilege escalation
Missing server-side authorization	High	Unauthorized resource access
Proper role validation	Low/No Finding	Access is appropriately restricted

RECOMMENDATIONS

1. Perform authorization checks on the server side.
2. Validate the user's role for every protected resource.
3. Do not rely on hidden fields or client-side controls.
4. Use secure object-reference mechanisms.
5. Apply least-privilege principles.
6. Log and monitor unauthorized access attempts.
7. Test authorization controls regularly.

RESULT

The student portal's access-control mechanisms were assessed using authorized test accounts. The assessment demonstrated the importance of server-side authorization and role validation in preventing unauthorized access to protected resources.

Question 5: Automated Scanning and Manual Validation

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

AIM

To perform automated vulnerability assessment of an authorized web application using Nikto and OWASP ZAP or Wapiti, compare the findings, safely validate selected findings, and recommend appropriate remediation.

PROCEDURE

1. Confirm that the web application is an authorized laboratory target.
2. Perform web-server scanning using Nikto.
3. Perform automated web-application scanning using OWASP ZAP or Wapiti.
4. Compare the findings reported by both tools.

5. Select at least three findings for safe manual validation and classify each as confirmed, false positive, or requiring further investigation.

COMMANDS

Nikto nikto -h

http://<target-ip> Nmap

Supporting Scan nmap -

sC -sV <target-ip>

OWASP ZAP

1. Open OWASP ZAP.
2. Configure the authorized target.
3. Browse the application to populate the site tree.
4. Review passive-scan alerts.
5. Run the authorized automated scan.
6. Review and export the results.

WAPITI

Where Wapiti is approved for the laboratory:

wapiti -u http://<authorized-target>

COMPARATIVE ANALYSIS

Finding	Nikto	ZAP/Wapiti	Validation	Severity
Information disclosure	Detected/Not detected	Detected/Not detected	Confirmed/False Positive	Low/Medium
Security misconfiguration	Detected/Not detected	Detected/Not detected	Confirmed/False Positive	Medium
Authentication issue	Detected/Not detected	Detected/Not detected	Confirmed/Needs Investigation	High
Injection-related issue	Detected/Not detected	Detected/Not detected	Confirmed/Needs Investigation	High

MANUAL VALIDATION

At least three findings should be reviewed manually.

Finding 1

Name: <finding>

Tool: <Nikto/ZAP/Wapiti>

Observation: <actual evidence>

Validation Status: Confirmed / False Positive / Further Investigation

Severity: <Low/Medium/High/Critical>

Impact: <impact>

Recommendation: <remediation>

Finding 2

Name: <finding>

Tool: <tool>

Observation: <actual evidence>

Validation Status: Confirmed / False Positive / Further Investigation

Severity: <severity>

Impact: <impact>

Recommendation: <remediation>

CATEGORIZATION

Category	Examples
Misconfiguration	Missing security headers, directory listing
Information Disclosure	Server/version disclosure, exposed files
Authentication	Weak authentication configuration
Injection	Improper input validation
Session Management	Weak session configuration

SEVERITY GUIDELINES

Critical: Vulnerability could result in severe compromise or major unauthorized access.

High: Vulnerability could lead to significant unauthorized access, data exposure, or system compromise.

Medium: Vulnerability presents a meaningful security weakness but normally requires additional conditions.

Low: Vulnerability has limited direct impact but should still be corrected as part of security hardening.

RECOMMENDATIONS

1. Patch outdated software and dependencies.
2. Remove unnecessary files and directories.
3. Configure secure HTTP headers.
4. Implement strong authentication and session management.
5. Validate and sanitize application input.
6. Apply server-side authorization controls.
7. Restrict administrative interfaces.
8. Disable unnecessary services.
9. Monitor security events and suspicious requests.
10. Perform regular automated and manual security assessments.

RESULT

Automated vulnerability assessment was performed using authorized security tools. Findings from multiple scanners were compared and selected findings were manually reviewed to distinguish confirmed vulnerabilities from false positives or issues requiring further investigation.